A1: Symmetries and conservation laws

Assignment handout

Training dynamics, a self-study course · written by Claude for Fan Pu

October 29, 2026

about 5 hours · CPU only · all six checks together run in about 15 seconds

1 Overview

Gradient flow on the anchor $f_{a,b}(x) = abx$, $L = \frac{1}{2}(ab-1)^2$, never leaves the curve $a^2 - b^2 = \text{const}$ (A0). This assignment asks *why*, how far the reason reaches, and what it buys. The why is a symmetry: the loss is unchanged when one layer is scaled up and the other down. What it buys is a solved model: the endpoint, the learning curve and the time to learn have closed forms, and the same reduction solves a two-layer linear network with matrices, mode by mode. Real training breaks the law (finite steps, momentum, minibatches, weight decay), and *modified gradient flow* predicts by how much. A0 supplied the flow, the phase portrait and the one-layer mode decomposition; A2 (weight decay), A3 (finite steps) and A5 (lazy vs rich) start from this assignment's conserved quantities.

What you will derive

1. The Noether-style constraint on the gradient and the conserved quantity it implies, for any loss of ab (Problem (`noether`))
2. The exact endpoint of the flow on $ab = s$ from the conserved imbalance (Problem (`endpoint`))
3. The logistic learning curve of a balanced mode and the time to learn it (Problem (`logistic`))
4. The matrix law $\frac{d}{dt}(W_1 W_1^\top - W_2^\top W_2) = 0$ and its exact one-step violation under GD (Problem (`matrix-law`))
5. Saxe's decomposition of a two-layer linear network into independent anchors (Problem (`saxe`))
6. How GD, momentum, minibatch SGD and weight decay change c, to first order in η (Problem (`broken`))

What you will implement

1. `noether.py`, `endpoint.py`, `logistic.py` in `symmetries/`: the symmetry, the endpoint, the learning curve (Problems (`noether`)–(`logistic`))
2. `two_layer.py`, `saxe.py`: gradients and the matrix law, Saxe's prediction (Problems (`matrix-law`), (`saxe`))
3. `drift.py`: the exact and first-order breaking of the law (Problem (`broken`))

What you will run

1. Six check scripts, one per problem, each comparing a simulation with your derivation
2. A short reading of Simon et al. on where conservation laws sit in a theory of deep learning (Problem (`framing`))

Setup. From the course root: `uv run pytest a1-symmetries` (all 48 tests fail with `NotImplementedError` until you fill in the stubs). Checks run as `uv run python a1-symmetries/scripts/check_<name>.py` and write PNGs and their section of `a1-symmetries/out/results.json`; compare with `uv run python -m tdyn.results a1-symmetries/out/results.json a1-symmetries/solutions/results.json`. Three checks have a `FORMULAS` block at the top where you type a derived formula. Worked solutions: `solutions/solutions.pdf`. **Time box.** If a Derive part takes more than 15 minutes with all hints, read that part in

`solutions/solutions.pdf`, close it, and redo the part from a blank page before moving on. The solutions are part of the method, not a failure. Rusty on the math? Parts point into the primer `m0-math-primer/handout.pdf` at the point of use, as in (primer §ODE.4, logistic) its first page is a one-page self-diagnostic.

Notation. As in A0: $\theta = (a, b)$, residual $r = ab - 1$ (or $ab - s$ for a target $s > 0$), product $p = ab$, imbalance $c = a^2 - b^2$, step size η, weight decay γ in the convention $L_\gamma = L + \gamma(a^2 + b^2)$, momentum β. $\langle u, v \rangle$ is the dot product.

2 Symmetry and conservation

A *continuous symmetry* of a loss $L(\theta)$ is a family of maps ψ_α, indexed by a real number α with ψ_1 the identity, such that $L(\psi_\alpha(\theta)) = L(\theta)$ for every θ and α. Its *generator* is the velocity of $\psi_\alpha(\theta)$ as α leaves the identity,

$$X(\theta) = \frac{\partial}{\partial \alpha} \psi_\alpha(\theta) \mid_{\alpha=1}, \tag{1}$$

a vector field on parameter space. A *conserved quantity* (or *constant of motion*) is a function $Q(\theta)$ that stays constant along every trajectory. Noether's theorem in physics says each continuous symmetry of a Lagrangian gives a conserved quantity; Kunin et al. prove the gradient-flow analogue for neural networks [D. Kunin et al., 2021, §3–4, Theorem 1]: differentiate the symmetry at the identity to get a constraint on the gradient, then project the flow onto the generator. The anchor's symmetry is the *rescaling*

$$\psi_\alpha(a, b) = \left(\alpha a, \frac{b}{\alpha}\right), \tag{2}$$

which leaves ab, so the network function and every loss built from it, unchanged; for ReLU networks the same map acts on each hidden unit's incoming and outgoing weights [J. M. L. Ang, 2026, §2.1] [S. S. Du et al., 2018, Theorem 2.1].

Example (`scale-by-hand`): The same recipe for a different symmetry

$L(\theta) = -\theta_2/\|\theta\|$ on $\mathbb{R}^2$ depends only on the direction of θ (like weights feeding a normalization layer). Its symmetry is *scaling*, $\psi_\alpha(\theta) = \alpha\theta$, with generator $X(\theta) = \theta$. Differentiating $L(\alpha\theta) = L(\theta)$ at $\alpha = 1$ gives $\langle \nabla L(\theta), \theta \rangle = 0$: the gradient is always tangent to the circle through θ. Under gradient flow, $\frac{d}{dt}\|\theta\|^2 = 2\langle \theta, \dot{\theta} \rangle = -2\langle \theta, \nabla L \rangle = 0$, so $\|\theta\|^2$ is conserved. One GD step tells another story. At $\theta = (1, 0)$: $\nabla L = (0, -1)$; with $\eta = 0.5$, $\theta' = (1, 0.5)$ and $\|\theta'\|^2 = 1.25$. Because the step is orthogonal to θ, Pythagoras gives $\|\theta'\|^2 = \|\theta\|^2 + \eta^2\|\nabla L\|^2$ exactly: every finite step *increases* the conserved quantity, by an amount of order η^2 [D. Kunin et al., 2021, §6, "scale dynamics"].

Problem (`noether`): Rescaling symmetry and the conserved imbalance [Derive + Implement + Check, 40 min] **[you]**

(a) **[Derive]** Differentiate $L(\psi_\alpha(\theta)) = L(\theta)$ in α at $\alpha = 1$ for Equation 2 (primer §LA.6, symmetries). Use the constraint to find a quadratic $Q(a, b)$ with $\dot{Q} = 0$ along gradient flow, and state for which losses $L(a, b)$ your argument holds (it should cover any per-example loss of $f_{a,b}$ on any dataset). **Deliverable**: the generator $X(a, b)$; the constraint, as the equation $\langle \nabla L(\theta), X(\theta) \rangle = \ldots$; $Q(a, b)$; the class of losses, stated as a condition on L (a form $L(a, b) = \ldots$ that guarantees the symmetry).

Hints, gentle to strong: (1) Chain rule on $\alpha \mapsto L(\psi_\alpha(\theta))$ at $\alpha = 1$ (primer §LA.6). (2) Look for Q with $\nabla Q \propto X$: then $\dot{Q} = \langle \nabla Q, \dot{\theta} \rangle = -\langle \nabla Q, \nabla L \rangle$ vanishes by the constraint. (3) First line of the solution: "The generator of Equation 2 is $X(a,b) = \partial_\alpha(\alpha a, b/\alpha)|_{\alpha=1} = (a, -b)$."

(b) **[Derive]** Differentiate your constraint once more with respect to θ (Kunin et al. do this in §3) (primer §LA.6, differentiating an identity). Deduce what $\nabla^2 L(\theta) X(\theta)$ is at any minimum, and connect it to the zero eigenvalue you found in A0's Problem (`phase-portrait`). **Deliverable**: the identity in the form $\nabla^2 L(\theta) X(\theta) = M \nabla L(\theta)$ with M a constant 2×2 matrix; the sentence "at a minimum, $X(\theta)$ is an eigenvector of $\nabla^2 L$ with eigenvalue ...", and which eigenvector of A0's Problem (`phase-portrait`) (b) it is.
Hints, gentle to strong: (1) The constraint holds for every θ, so its gradient in θ vanishes too: product rule on $\langle \nabla L(\theta), X(\theta) \rangle$ (primer §LA.6). (2) $\nabla_\theta \langle \nabla L, X \rangle = \nabla^2 L\, X + (\partial X / \partial \theta)^\top \nabla L$, and here $\partial X / \partial \theta$ is a constant diagonal matrix. (3) First line of the solution: "Differentiate the constraint $\langle \nabla L(\theta), X(\theta) \rangle = 0$ with respect to θ."

(c) **[Derive]** *Small initialization gives balanced layers*
[S. S. Du et al., 2018, §2.1, after Corollary 2.1]. For a_0, b_0 i.i.d. $\mathcal{N}(0, \sigma^2)$, compute $\sqrt{\mathbb{E}[c^2]}$ (primer §PR.1, Gaussian moments). Then use $a_\infty^2 - b_\infty^2 = c$ to find the gap $a_\infty - b_\infty$ to leading order in small c on the positive branch of $ab = 1$. **Deliverable**: $\sqrt{\mathbb{E}[c^2]}$ as a function of σ; $a_\infty - b_\infty$ to first order in c. Type both into the `FORMULAS` block of `scripts/check_noether.py`.
Hints, gentle to strong: (1) Expand c^2 and use Gaussian moments, $\mathbb{E}[a^4] = 3\sigma^4$, and independence (primer §PR.1). (2) Factor $a_\infty^2 - b_\infty^2 = (a_\infty - b_\infty)(a_\infty + b_\infty)$ and find $a_\infty + b_\infty$ to zeroth order in c (primer §TAY.4, §ALG.1). (3) First line of the solution: "With a_0, b_0 i.i.d. $\mathcal{N}(0, \sigma^2)$, $\mathbb{E}[a^4] = 3\sigma^4$, so $\mathbb{E}[c^2] = \mathbb{E}[a^4] - 2\mathbb{E}[a^2]\mathbb{E}[b^2] + \mathbb{E}[b^4]$."

(d) **[Implement]** In `symmetries/noether.py`, for arrays of points of shape `(..., 2)`:

`def rescale(theta, alpha)` · `def generator(theta)` · `def conserved(theta)`

- **`rescale`** $\psi_\alpha(\theta)$ by Equation 2.
- **`generator`** $X(\theta)$ by Equation 1, your (a).
- **`conserved`** your $Q(\theta)$, shape `(...)`.

To test your implementation, run `uv run pytest a1-symmetries -k test_noether` (the adapters `run_rescale`, `run_generator`, `run_conserved` in `tests/adapters.py` are pre-wired to these signatures; edit them only if yours differ).

(e) **[Check]** Run `scripts/check_noether.py`. Expected: $|\langle \nabla L, X \rangle|$ at round-off for three losses of ab, order one for $f = (ab + 0.3a)x$ (a skip connection); your Q constant to 10^{-9} along those three flows, not along the skip-connection flow or GD at $\eta = 0.1$; the Hessian identity at round-off; `noether.png` (right) on your formulas for $\sigma \leq 0.3$.

3 Where the flow lands, and how fast

Problem (`endpoint`): Where gradient flow lands [Derive + Implement + Check, 25 min] **[you]**

Gradient flow on $L = \frac{1}{2}(s - ab)^2$ with target $s > 0$ ends on the curve $ab = s$ unless it ends at the origin (A0). Ang's post does $s = 1$ [J. M. L. Ang, 2026, §2.1].

(a) **[Derive]** Combine conservation with $a_\infty b_\infty = s$ to find a_∞^2 and b_∞^2 (primer §ALG.1, two equations). Decide which branch of $ab = s$ the flow reaches from (a_0, b_0), including $c = 0$, and

argue why the sign of a (if $c > 0$) or of b (if $c < 0$) cannot change. **Deliverable**: (a_∞, b_∞) in closed form for every start, with one sign rule that covers all cases.
Hints, gentle to strong: (1) Two equations in a_∞^2, b_∞^2: their difference is c, their product s^2; a quadratic in $A = a_\infty^2$ (primer §ALG.1). (2) For the signs: if $c > 0$ then $a^2 = c + b^2 \geq c > 0$ for all t, so a never passes through zero (primer §ODE.1). (3) First line of the solution: "Write $A = a_\infty^2$. Conservation gives $b_\infty^2 = A - c$ and the endpoint gives $a_\infty^2 b_\infty^2 = s^2$, so $A^2 - cA - s^2 = 0$."

(b) **[Implement]** In `symmetries/endpoint.py`:

```
def flow_endpoint(a0, b0, s=1.0) -> np.ndarray
```

- **returns** (a_∞, b_∞), shape `(2,)`; the origin for $a_0 = -b_0$.

To test your implementation, run `uv run pytest a1-symmetries -k test_endpoint` (the adapters `run_flow_endpoint` in `tests/adapters.py` are pre-wired to these signatures; edit them only if yours differ).

(c) **[Check]** Run `scripts/check_endpoint.py`: 450 random starts and $s \in \{0.5, 1, 2\}$, integrated to $t = 60$. Expected: your endpoints within 10^{-9} of the integrated ones (`endpoint.png`). It also starts at distance ε from the line $a = -b$; note in `notes.md` how the time to leave the origin grows with ε (A0's saddle eigenvalues explain it).

On the line $a = b > 0$ (so $c = 0$), $u = ab = a^2$ is the only variable left. Saxe, McClelland and Ganguli solve exactly this one-dimensional problem [A. M. Saxe et al., 2014, §1.3, eqs. (8)–(12)]; it is the learning curve of a single mode of a deep linear network (Problem (`saxe`)). Define the *time to learn* $t_f(u_0, s)$ as the first time at which u reaches the fraction f of its target,

$$u(t_f) = fs, \quad 0 < u_0 < fs, \quad t_{1/2} = t_f \text{ at } f = \frac{1}{2}. \tag{3}$$

Problem (`logistic`): Balanced learning is logistic [Derive + Implement + Check + Predict, 50 min] **[you]**

(a) **[Derive]** Show that $a_0 = b_0 > 0$ implies $a(t) = b(t)$ for all t, and derive a closed ODE for $u = ab$. It is the *logistic equation* (growth rate proportional to u, braked as u approaches its ceiling). Solve it by separation of variables and partial fractions (primer §ODE.4, logistic). **Deliverable**: the ODE $\dot{u} = F(u)$; $u(t)$ in closed form given u_0 and s.
Hints, gentle to strong: (1) Show $q = a - b$ obeys a linear ODE $\dot{q} = k(t)q$, so $q_0 = 0$ forces $q \equiv 0$ by uniqueness (primer §ODE.1); then differentiate $u = ab$ by the product rule. (2) Separate variables and use partial fractions (primer §ODE.4, §ALG.3). (3) First line of the solution: "For $L = \frac{1}{2}(s - ab)^2$: $\dot{a} = b(s - ab)$, $\dot{b} = a(s - ab)$, so $d/dt(a - b) = -(a - b)(s - ab)$."

(b) **[Derive]** From (a), compute $t_f(u_0, s)$, then its leading behaviour for $u_0 \ll s$ at $f = \frac{1}{2}$ (primer §ODE.4, time between two levels), (primer §TAY.4, leading order). Describe the shape of $u(t)$ for $u_0 = 10^{-6}$: how long is the *plateau* (the stretch where u and the loss barely move) compared with the *drop* (the time from $u = 0.1s$ to $u = 0.9s$)? **Deliverable**: t_f exactly; $t_{1/2}$ to leading order; how plateau length and drop length each scale with u_0 and s.
Hints, gentle to strong: (1) Partial fractions (primer §ODE.4): $\frac{1}{u(s-u)} = \frac{1}{s}\left(\frac{1}{u} + \frac{1}{s-u}\right)$. (2) Write the time from u_1 to u_2 as $\int_{u_1}^{u_2} du/F(u)$. (3) For the drop, both endpoints are fixed fractions of s, so u_0 drops out.

(c) **[Implement]** In `symmetries/logistic.py`, broadcasting over numpy arrays:

`def balanced_product(u0, s, t)` · `def time_to_learn(u0, s, frac=0.5)`

- **`balanced_product`** $u(t)$ from (a); finite for any $t \geq 0$ (rewrite it if e^{2st} would overflow).
- **`time_to_learn`** $t_f(u_0, s)$ with $f =$ `frac`, from (b).

To test your implementation, run `uv run pytest a1-symmetries -k test_logistic` (the adapters `run_balanced_product`, `run_time_to_learn` in `tests/adapters.py` are pre-wired to these signatures; edit them only if yours differ).

(d) **[Check]** Run `scripts/check_logistic.py`. It integrates the full 2-D flow for $s \in \{0.5, 1, 2, 4\}$ and 15 values of u_0 from 10^{-8} to 10^{-1} and finds when ab crosses $s/2$. Expected: your $t_{1/2}$ within 10^{-8} relative, and the slope of $t_{1/2}$ against $\ln(1/u_0)$ equal to your prediction from (b) to 1% (`logistic.png`, left).

(e) **[Predict]** Before looking at the right panel, write in `notes.md`: at fixed $a_0 b_0 = 10^{-4}$ and $s = 1$, does an *unbalanced* start, $c = a_0^2 - b_0^2 > 0$, learn faster or slower than the balanced one, and roughly from which c on does it matter? Then derive $\dot{p}$ for general c (the post's §2.1 has it) and explain the panel. **Deliverable**: the prediction; $\dot{p}$ as a function of p, c, s; two sentences.

4 Two-layer linear networks

Replace the scalars by matrices: $\hat{y} = W_2 W_1 x$ with $W_1 \in \mathbb{R}^{m \times d_{\text{in}}}$, $W_2 \in \mathbb{R}^{d_{\text{out}} \times m}$, hidden width m, and with $\Sigma_{xx} = \mathbb{E}[xx^\top]$, $\Sigma_{yx} = \mathbb{E}[yx^\top]$ as in A0,

$$L(W_1, W_2) = \frac{1}{2}\mathbb{E}\|W_2 W_1 x - y\|^2 = \frac{1}{2}\operatorname{tr}(W \Sigma_{xx} W^\top) - \operatorname{tr}(W \Sigma_{yx}^\top) + \text{const}, \quad W = W_2 W_1. \tag{4}$$

L depends on (W_1, W_2) only through W, so $(W_1, W_2) \to (AW_1, W_2 A^{-1})$ is a symmetry for every invertible $m \times m$ matrix A (m^2 parameters instead of one), and the conserved quantity is a matrix,

$$D = W_1 W_1^\top - W_2^\top W_2 \in \mathbb{R}^{m \times m}. \tag{5}$$

Du, Hu and Lee prove that gradient flow keeps D fixed for any two consecutive linear layers inside any network [S. S. Du et al., 2018, §2.1, Theorem 2.2]; Simon et al. list it among the "simple empirical laws" of deep learning [J. Simon et al., 2026, §2.3, p. 13].

Problem (`matrix-law`): The matrix conservation law [Derive + Implement + Check, 45 min] **[you]**

(a) **[Derive]** With $G = \nabla_W L$ the gradient of Equation 4 with respect to the product, find $\nabla_{W_1} L$ and $\nabla_{W_2} L$ by the chain rule (perturb $W_1 \to W_1 + E$ and keep the first-order term of L) (primer §LA.5, matrix calculus). **Deliverable**: G, $\nabla_{W_1} L$, $\nabla_{W_2} L$ as matrix products.
Hints, gentle to strong: (1) Matrix gradients by perturbation: write the first-order change of L as $\langle \cdot, E \rangle$ using the cyclic trace (primer §LA.5). (2) Perturbing W_1 by E perturbs $W = W_2 W_1$ by $W_2 E$, and the change of L is $\langle G, W_2 E \rangle$. (3) First line of the solution: "$L(W) = \frac{1}{2}\operatorname{tr}(W \Sigma_{xx} W^\top) - \operatorname{tr}(W \Sigma_{yx}^\top)$ has $G = \nabla_W L = W \Sigma_{xx} - \Sigma_{yx}$."

(b) **[Derive]** Show $\dot{D} = 0$ under gradient flow, using only that L depends on $W_2 W_1$ through some G. Then derive it a second way, Noether-style: take $A = I + \varepsilon M$ for an arbitrary M and differentiate at $\varepsilon = 0$; which matrix does "for all M" force to vanish? For $m = d_{\text{in}} = d_{\text{out}} = 1$, recover Problem (`noether`) (primer §LA.5, products and "for all M"), (primer §LA.6,

symmetries). **Deliverable**: the two-line proof that $\dot{D} = 0$; the constraint as an $m \times m$ matrix identity $K = 0$, with K written in W_1, W_2, $\nabla_{W_1} L$, $\nabla_{W_2} L$.
Hints, gentle to strong: (1) Product rule: $\dot{D} = \dot{W}_1 W_1^\top + W_1 \dot{W}_1^\top - \dot{W}_2^\top W_2 - W_2^\top \dot{W}_2$; substitute the gradients from (a) (primer §LA.5). (2) For the Noether route, $(I + \varepsilon M)^{-1} = I - \varepsilon M + O(\varepsilon^2)$; the derivative at $\varepsilon = 0$ has the form $\mathrm{tr}(MK)$, and $\mathrm{tr}(MK) = 0$ for all M forces $K = 0$ (primer §LA.5, §LA.6). (3) First line of the solution: "Under gradient flow $\dot{W}_1 = -W_2^\top G$, $\dot{W}_2 = -G W_1^\top$."

(c) **[Derive]** Take one GD step, $W_{i'} = W_i - \eta \nabla_{W_i} L$. Compute $D' - D$ *exactly* (no expansion in η) (primer §TAY.3, a conserved quantity under one step): the first-order terms cancel; write what remains in terms of $\nabla_{W_1} L$ and $\nabla_{W_2} L$. **Deliverable**: $D' - D$ in terms of η, $\nabla_{W_1} L$, $\nabla_{W_2} L$; its scalar case as a function of c, r, η.
Hints, gentle to strong: (1) Expand the products exactly and collect powers of η, as in the scalar worked example of primer §TAY.3. (2) The η^1 terms of $W_{1'} W_{1'}^\top$ and of $W_{2'}^\top W_{2'}$ are equal; check it with your (a). (3) First line of the solution: "With $g_i = \nabla_{W_i} L$: $W_{1'} W_{1'}^\top = W_1 W_1^\top - \eta(g_1 W_1^\top + W_1 g_1^\top) + \eta^2 g_1 g_1^\top$."

(d) **[Implement]** In `symmetries/two_layer.py`:

`def two_layer_grads(W1, W2, Sxx, Syx) · def imbalance(W1, W2) · def gd_imbalance_change(W1, W2, Sxx, Syx, eta)`

- **`two_layer_grads`** $\left(\nabla_{W_1} L, \nabla_{W_2} L\right)$ from (a), matrix products only.
- **`imbalance`** D by Equation 5.
- **`gd_imbalance_change`** your exact $D' - D$ from (c), without taking the step.

To test your implementation, run `uv run pytest a1-symmetries -k test_two_layer` (the adapters `run_two_layer_grads`, `run_imbalance`, `run_gd_imbalance_change` in `tests/adapters.py` are pre-wired to these signatures; edit them only if yours differ).

(e) **[Check]** Run `scripts/check_matrix_law.py` ($d_{\text{in}} = 4$, $m = 3$, $d_{\text{out}} = 2$, correlated inputs). Expected: $\|D(t) - D(0)\|_F$ below 10^{-11} along RK4 gradient flow; under GD every step's change equal to your formula to round-off; the accumulated drift at $t = 10$ growing like η^p (`matrix_law.png`). Predict p from (c) before you look: how many steps does it take to reach $t = 10$?

4.1 Saxe's mode decomposition

Now let the inputs be *whitened*, $\Sigma_{xx} = I$, and take the *singular value decomposition* (SVD) $\Sigma_{yx} = \sum_\alpha s_\alpha \boldsymbol{u}_\alpha \boldsymbol{v}_\alpha^\top = U S V^\top$ with singular values $s_1 \geq s_2 \geq \ldots > 0$ and orthonormal singular vectors $\boldsymbol{u}_\alpha$ (outputs), $\boldsymbol{v}_\alpha$ (inputs), written in bold to keep them apart from the scalar u of Problem (`logistic`). A0 Problem (`modes`) showed that one layer learns all modes at the same rate. Saxe et al. show that two layers learn them *one at a time* [A. M. Saxe et al., 2014, §1.1–1.3]. The key object is the *aligned, balanced* initialization: pick any $m \times r$ matrix R with orthonormal columns (r the number of modes) and set $W_1 = R\,\mathrm{diag}\big(\sqrt{u_{0\alpha}}\big)\,V^\top$, $W_2 = U\,\mathrm{diag}\big(\sqrt{u_{0\alpha}}\big)R^\top$. Then (part (a) below) the product stays of the form

$$W(t) = W_2(t) W_1(t) = \sum_\alpha u_\alpha(t)\, \boldsymbol{u}_\alpha \boldsymbol{v}_\alpha^\top, \tag{6}$$

and each *mode strength* $u_\alpha(t) = \boldsymbol{u}_\alpha^\top W(t) \boldsymbol{v}_\alpha$ evolves on its own.

Example (`svd-by-hand`): Modes of a 2×2 correlation

$\Sigma_{yx} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$ is symmetric with eigenvectors $(1,1)/\sqrt{2}$ and $(1,-1)/\sqrt{2}$, eigenvalues 3 and 1. Both are positive, so its SVD has $\boldsymbol{u}_\alpha = \boldsymbol{v}_\alpha$: $s_1 = 3$ along $(1,1)/\sqrt{2}$, $s_2 = 1$ along $(1,-1)/\sqrt{2}$. A two-layer net learns the map "add the two inputs" (s_1) before "subtract them" (s_2); part (b) below says how much later.

Problem (`saxe`): Deep linear networks learn mode by mode [Derive + Implement + Check, 60 min] **`[you]`**

(a) **`[Derive]`** Change variables to $\overline{W}_1 = W_1 V$ and $\overline{W}_2 = U^\top W_2$ (extend U, V to square orthogonal matrices). From the aligned balanced initialization, show that $\overline{W}_1$ and $\overline{W}_2$ keep their initial structure, that Equation 6 holds, and that each pair (a_α, b_α), defined by column α of $\overline{W}_1$ being $a_\alpha r_\alpha$ and row α of $\overline{W}_2$ being $b_\alpha r_\alpha^\top$ (r_α the columns of R), follows the anchor with target s_α. **Deliverable**: the ODEs for (a_α, b_α); one sentence naming the term that would couple two modes $\alpha \neq \beta$ and why it vanishes.
Hints, gentle to strong: (1) Rotate both sides by the singular vectors so that Σ_{yx} becomes the diagonal S (primer §LA.4); the ansatz argument is primer §ODE.1. (2) Use your gradients from Problem (`matrix-law`) with $\Sigma_{xx} = I$; verify that the ansatz is consistent (its velocity has the same structure) and invoke uniqueness of ODE solutions. (3) First line of the solution: "With $\Sigma_{xx} = I$ the flow is $\dot{W}_1 = W_2^\top(\Sigma_{yx} - W_2 W_1)$, $\dot{W}_2 = (\Sigma_{yx} - W_2 W_1) W_1^\top$."

(b) **`[Derive]`** With Problem (`logistic`), give $u_\alpha(t)$, and the ratio $t_{1/2,\alpha}/t_{1/2,\beta}$ of half-times for two modes from the same small u_0, to leading order (primer §ALG.5, time scales). **Deliverable**: $u_\alpha(t)$; the ratio, exactly and to leading order, as a formula in s_α, s_β (and u_0).
Hints, gentle to strong: (1) Each mode is a balanced anchor with target s_α: reuse Problem (`logistic`) (primer §ODE.4). (2) For small u_0, $t_{1/2} \approx \frac{1}{2s}\ln(s/u_0)$; to leading order keep the $1/s$ outside the logarithm (primer §TAY.4, §ALG.5). (3) First line of the solution: "The init is balanced ($a_\alpha = b_\alpha = \sqrt{u_{0\alpha}}$), so each mode is Problem (`logistic`)."

(c) **`[Implement]`** In `symmetries/saxe.py` ($\Sigma_{xx} = I$; modes are singular triples with $s_\alpha > 10^{-10} s_1$, in `np.linalg.svd` order; `u0` a scalar or one value per mode):

`def aligned_init(Syx, u0, hidden)` · `def saxe_prediction(Syx, u0, t)` · `def effective_u0(W1, W2, Syx)`

- **`aligned_init`** (W_1, W_2) as above, with any valid R.
- **`saxe_prediction`** $W(t)$ by Equation 6 and (b).
- **`effective_u0`** the per-mode u_0^{eff} of (d).

To test your implementation, run `uv run pytest a1-symmetries -k test_saxe` (the adapters `run_aligned_init`, `run_saxe_prediction`, `run_effective_u0` in `tests/adapters.py` are pre-wired to these signatures; edit them only if yours differ).

(d) **`[Derive]`** Real networks start from small *random* weights, not aligned ones. For mode α let $x = W_1 \boldsymbol{v}_\alpha \in \mathbb{R}^m$ and $y = W_2^\top \boldsymbol{u}_\alpha \in \mathbb{R}^m$, so $u_\alpha = x^\top y$. While all weights are small, drop the terms cubic in the weights from the flow and solve for $x(t)$, $y(t)$ (primer §ODE.6, linear systems). Match the result with the early phase of (b), $u(t) \approx u_0 e^{2st}$. **Deliverable**: u_0^{eff} for mode α as a function of $W_1(0)$, $W_2(0)$, $\boldsymbol{u}_\alpha$, $\boldsymbol{v}_\alpha$.

Hints, gentle to strong: (1) Dropping the cubic terms leaves a linear system; solve it through its eigenvectors (primer §ODE.6). (2) The linearized system is $\dot{x} = s_\alpha y$, $\dot{y} = s_\alpha x$; look at $x + y$ and $x - y$, and use $x^\top y = \frac{1}{4}(\|x+y\|^2 - \|x-y\|^2)$. (3) First line of the solution: "Linearize: $\dot{x} = \dot{W}_1 \boldsymbol{v}_\alpha = W_2^\top (\Sigma_{yx} - W)\boldsymbol{v}_\alpha \approx W_2^\top \Sigma_{yx} \boldsymbol{v}_\alpha = s_\alpha W_2^\top \boldsymbol{u}_\alpha = s_\alpha y$."

(e) **[Check]** Run `scripts/check_saxe.py`: $d_{\text{in}} = 8$, $d_{\text{out}} = 6$, $m = 6$, $s = (4, 2, 1, 0.5)$. Expected: from the aligned init your $W(t)$ within 10^{-7} of the integrated flow; from a random init of scale 10^{-3}, half-times within 5% of your prediction with u_0^{eff}, a small delay growing for weaker modes (competition between modes, Saxe et al. Fig. 3 right), and a loss made of plateaus and drops (`saxe.png`).

5 How the law breaks

Gradient flow is a model. Real training takes finite steps η, often with heavy-ball *momentum* $\theta_{k+1} = \theta_k - \eta \nabla L(\theta_k) + \beta(\theta_k - \theta_{k-1})$, on minibatches, with weight decay. In GD with weight decay, one step on the anchor reads

$$a_{k+1} = a_k - \eta(b_k r_k + 2\gamma a_k), \quad b_{k+1} = b_k - \eta(a_k r_k + 2\gamma b_k). \tag{7}$$

To track a slowly drifting quantity, find an ODE that the iterates follow more closely than gradient flow. *Backward error analysis* (A3 derives it) gives, for GD, the *modified gradient flow*

$$\dot{\theta} = -\nabla L(\theta) - \frac{\eta}{2} \nabla^2 L(\theta) \nabla L(\theta), \tag{8}$$

which matches GD to one order higher in η; time is $t = k\eta$. For heavy-ball momentum started at rest, Kunin et al. use the second-order *modified momentum flow*
[D. Kunin et al., 2021, §5 and App. G, with no damping]

$$\eta \frac{1+\beta}{2} \ddot{\theta} + (1-\beta)\dot{\theta} = -\nabla L(\theta), \quad \dot{\theta}(0) = 0, \tag{9}$$

again with $t = k\eta$. To leading order it is gradient flow slowed by $1 - \beta$, i.e. with effective step $\eta/(1-\beta)$. Kunin et al. combine these with the symmetry constraints and check the result on VGG-16 [D. Kunin et al., 2021, §6, Fig. 5]; here the anchor gives closed forms.

Problem (`broken`): Finite steps, momentum, minibatches and weight decay [Derive + Implement + Check, 75 min] **[you]**

(a) **[Derive]** From Equation 7, compute c_{k+1} exactly in terms of c_k, r_k, η, γ (primer §TAY.3, one GD step). Separately, for gradient flow on L_γ find $\dot{c}$ and solve it (primer §ODE.3, linear decay). (Ang's post states the $\gamma = 0$ step [J. M. L. Ang, 2026, §3.1]; the brief writes weight decay as $-\gamma a$, a factor 2 apart from this course's convention.) **Deliverable**: c_{k+1}; $c(t)$ under the flow with weight decay (type it into `wd_imbalance` in `scripts/check_drift.py`); for $\gamma = 0$, the condition on ηr_k, as an inequality, under which $|c|$ cannot grow.
Hints, gentle to strong: (1) Expand $a_{k+1}^2 - b_{k+1}^2$ exactly, as in primer §TAY.3; the flow part is a linear ODE (primer §ODE.3). (2) Write the step as $a' = (1 - 2\eta\gamma)a - \eta b r$; the cross terms cancel because $ab - ba = 0$. (3) First line of the solution: "Write Equation 7 as $a' = (1 - 2\eta\gamma)a - \eta b r$, $b' = (1 - 2\eta\gamma)b - \eta a r$."

(b) **[Derive]** Along Equation 8 ($\gamma = 0$), compute $\dot{c}$ (primer §LA.6, the generator). Use the constraint from Problem (`noether`) to kill the ∇L term; for the correction term you need $\nabla^2 L\, \nabla L$ explicitly. **Deliverable**: $\dot{c}$ as a function of η, r, c.
Hints, gentle to strong: (1) $\dot{c} = \left\langle \nabla c, \dot{\theta} \right\rangle = 2\left\langle X, \dot{\theta} \right\rangle$ with $X = (a, -b)$ (primer §LA.6). (2) $\langle X, \nabla^2 L \nabla L \rangle = \langle \nabla^2 L\, X, \nabla L \rangle$ since $\nabla^2 L$ is symmetric; use your identity from Problem (`noether`) (b). (3) First line of the solution: "Along Equation 8, $\dot{c} = -2\langle X, \nabla L \rangle - \eta \langle X, \nabla^2 L \nabla L \rangle$ with $X = (a, -b)$."

(c) **[Derive]** Integrate (b) to first order in η: replace $r(t)$ by its gradient-flow value. Show that $\ln(c_\infty / c_0) \approx -\eta\, I(p_0, c_0)$ with $I = \int_0^\infty r(t)^2\, dt$, change variables from t to p using $\dot{p}$ from Problem (`logistic`) (e), and evaluate the integral in closed form (primer §ODE.3, time-varying rate), (primer §ALG.3, integrating along a trajectory). How does $|\ln(c_1/c_0)|$ after a single step scale with η, and how does the total $|\ln(c_\infty/c_0)|$ scale? **Deliverable**: $I(p_0, c)$ in closed form ($c \neq 0$); the two log-log slopes against η.
Hints, gentle to strong: (1) $dt = dp/\dot{p}$ (primer §ALG.3); p moves monotonically from p_0 to 1. (2) $\int dp/\sqrt{c^2 + 4p^2} = \frac{1}{2}\,\mathrm{arcsinh}(2p/|c|)$. (3) $\int p\, dp/\sqrt{c^2 + 4p^2} = \frac{1}{4}\sqrt{c^2 + 4p^2}$.

(d) **[Derive]** Repeat (b)–(c) for momentum, starting from Equation 9. Take the inner product with $(a, -b)$ to get an equation for c, keep terms to first order in η (primer §TAY.2, first order), and integrate over the whole run. **Deliverable**: the factor by which momentum β multiplies the first-order drift of $\ln c$ at a fixed η.
Hints, gentle to strong: (1) Differentiate c twice (primer §TAY.1, chain rule): $\left\langle (a, -b), \ddot{\theta} \right\rangle = \frac{1}{2}\ddot{c} - \left(\dot{a}^2 - \dot{b}^2\right)$. (2) To leading order $\dot{\theta} = -\nabla L/(1-\beta)$, and the run takes $1/(1-\beta)$ times longer in t than gradient flow takes. (3) The $\ddot{c}$ term integrates to $\dot{c}(\infty) - \dot{c}(0)$; both vanish.

(e) **[Derive]** *Minibatches.* Take n examples with $\frac{1}{n}\sum_i x_i^2 = 1$ and $y_i = x_i$, per-example loss $\frac{1}{2}(abx_i - y_i)^2$, and SGD on batches of B indices drawn with replacement. Show that the batch gradient is $m_B r(b, a)$ with m_B the batch mean of x_i^2, so it is still orthogonal to X, and find c_{k+1}. To first order in η, by what factor does SGD's drift exceed GD's? (primer §PR.2, sample means) **Deliverable**: c_{k+1} for SGD; the factor in terms of B and the variance of x_i^2 over the dataset (type it into `sgd_drift_factor`).
Hints, gentle to strong: (1) The batch mean of x_i^2 multiplies the GD step; the drift needs the mean of its square (primer §PR.2). (2) $\mathbb{E}[m_B^2] = (\mathbb{E} m_B)^2 + \mathrm{Var}(m_B)$, and $\mathrm{Var}(m_B) = \mathrm{Var}(x_i^2)/B$ for draws with replacement. (3) First line of the solution: "Per-example gradient: $\partial_a \frac{1}{2}(abx_i - x_i)^2 = bx_i^2 r$, so the batch gradient is $m_B r(b, a)$ with $m_B = \frac{1}{B}\sum_{i \in B} x_i^2$."

(f) **[Implement]** In `symmetries/drift.py`:

`def imbalance_after_step(a, b, eta, gamma=0.0) · def drift_integral(p0, c) · def predicted_log_drift(a0, b0, eta, beta=0.0)`

- **`imbalance_after_step`** c_{k+1} by your (a), as a formula; do not take the step.
- **`drift_integral`** your closed form $I(p_0, c)$ from (c).
- **`predicted_log_drift`** the first-order prediction of $\ln(c_\infty/c_0)$, from (c) and (d).

To test your implementation, run `uv run pytest a1-symmetries -k test_drift` (the adapters `run_imbalance_after_step`, `run_drift_integral`, `run_predicted_log_drift` in `tests/adapters.py` are pre-wired to these signatures; edit them only if yours differ).

(g) **[Check]** Run `scripts/check_drift.py` from $(a_0, b_0) = (0.3, 0.1)$ with η from 0.002 to 0.1. Expected: the one-step formula exact to round-off; $c(t)$ under weight decay on your formula; both log-log slopes within 0.05 of your (c); measured over predicted drift tending to 1 as $\eta \to 0$ (within 1% at $\eta = 0.002$) for GD and $\beta = 0.3, 0.5$; SGD's ratio on your factor within two

standard errors for $B = 1, 4, 16$ (`drift.png`). A ratio tending to a constant other than 1 means a wrong factor in (c) or (d).

Note: Every minibatch gradient obeys the same constraint, so gradient noise is orthogonal to the generator [D. Kunin et al., 2021, §5]: it is the *finite step* that breaks the law. Which way the drift pushes $|c|$ is a first glimpse of the implicit regularization of A3 and A4.

6 Framing

Problem (`framing`): Conservation laws in the larger map [Read, 20 min] **`[you]`**

Read Simon et al. [J. Simon et al., 2026] §2.1 "Linearization in the data" with Figure 1 (pp. 6–7) and the paragraph "Gradient flow conservation laws" in §2.3 (p. 13). Answer briefly in `notes.md`:

(a) Figure 1(a) assumes "task-aligned initialization" and whitened inputs. Which did `check_saxe.py` drop in its second run, and what did it cost, in numbers?

(b) Conservation laws are "weakly broken by SGD in predictable ways". Which breakings did you predict, to which order in η, and what in real training would your predictions miss?

(c) Attention scores depend on $W_Q W_K^\top$. Write the symmetry this gives and, with Problem (`matrix-law`) (b), the matrix conserved by gradient flow.

7 What to write down

In `notes.md`, three to five sentences: what a symmetry gives you under gradient flow and under GD; why small init gives balanced layers and a long plateau; what "a deep linear network is a bundle of anchors" means and when it fails; one question for a tutorial.

References

[1] D. Kunin *et al.*, "Neural Mechanics: Symmetry and Broken Conservation Laws in Deep Learning Dynamics." International Conference on Learning Representations (ICLR), 2021. arXiv:2012.04728. Offline: papers/kunin2021-neural-mechanics.pdf.

[2] J. M. L. Ang, "Training Dynamics." 2026. Science of Deep Learning: Taster Series, 16 October 2026. Offline: papers/ang2026-training-dynamics-post.md.

[3] S. S. Du *et al.*, "Algorithmic Regularization in Learning Deep Homogeneous Models: Layers are Automatically Balanced." Advances in Neural Information Processing Systems 31 (NeurIPS), 2018. arXiv:1806.00900. Offline: papers/du2018-algorithmic-regularization-homogeneous.pdf.

[4] A. M. Saxe *et al.*, "Exact Solutions to the Nonlinear Dynamics of Learning in Deep Linear Neural Networks." International Conference on Learning Representations (ICLR), 2014. arXiv:1312.6120. Offline: papers/saxe2014-exact-solutions-deep-linear.pdf.

[5] J. Simon *et al.*, "There Will Be a Scientific Theory of Deep Learning." 2026. arXiv:2604.21691. Offline: papers/simon2026-scientific-theory-deep-learning.pdf.